Byte Latent Transformer — Akhil Kagithapu, Srikar Karra, Laerdon Kim, Ezra Koreen

The **Byte Latent Transformer (BLT)** is a transformer architecture which seeks to overcome challenges endemic to traditional tokenization-based LLMs. Today's LLMs, while powerful, cannot dynamically allocate compute at a token level to more difficult text generation problems nor do they have character-level awareness.

BLT solves this by instead performing inference on "entropy patches," which are collections of bytes which contain some information approximately equal in every patch. When a byte's entropy, as calculated by some small byte LM, rises above some threshold, we begin a new patch.

To contextualize our experiments on byte and patch representations, we provide a brief primer: byte-level embeddings are produced as an addition from embedding the byte itself with added contextual information from hashing a byte-gram of varying length to an embedding lookup table, then adding these embeddings to the byte representation, and finally, normalizing over all these addends, enriching the representation with context.

Secondly, the patch representations are initially built by max-pooling the byte embeddings within each patch. Then, we send these patches through a latent transformer, which refines the patch representations.

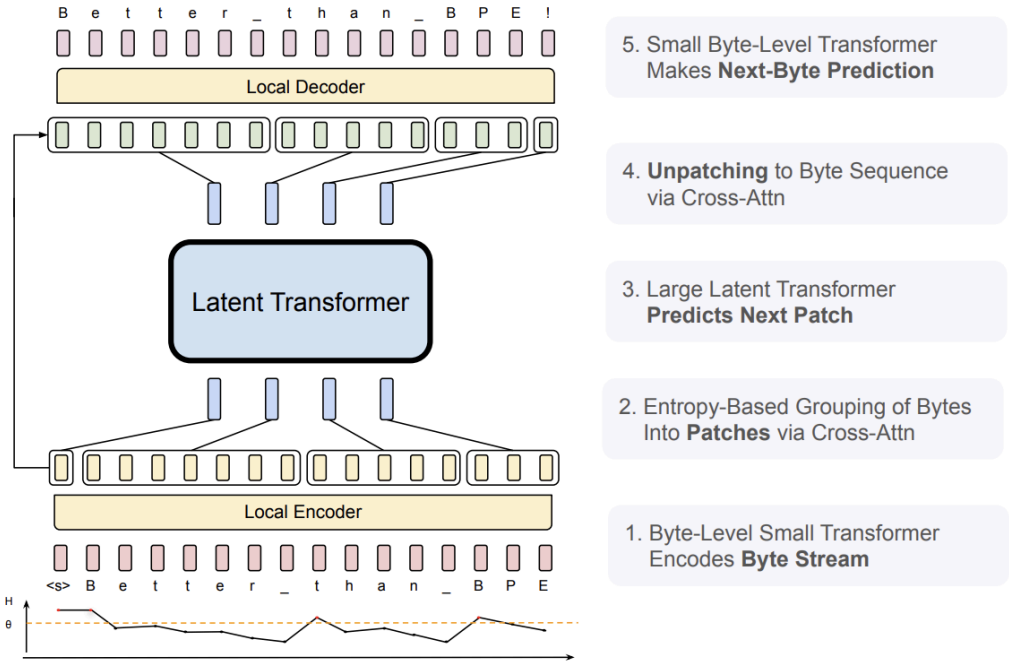


Figure 2 from Byte Latent Transformer depicting the full architecture.

For this paper, we sought to reproduce the scaling law of the BLT which demonstrates that the model is capable of reaching a lower validation BPB than the original architecture within a fixed FLOP budget. This is demonstrated in Figure 1 of the paper, the main evidence claiming that entropy patches scale better.

Our selection of this figure was also related to the feasibility of the project—in order to reproduce claims about superior downstream performance of the Byte Latent Transformer, i.e. on tasks like BLEU or HellaSwag we would need to train models at an infeasible scale. Our current models, at so few parameters, we anticipate would barely pass above chance.

In addition to this, we anticipated major challenges to working at such small parameter scales. As such, we also set aside compute to perform experiments of our own, which center the following:

- Varying distribution of parameters across components of the multi-transformer architecture

- Improving byte representations: Trying different context byte-gram configurations
- Improving patch representations: Adding sinusoidal embeddings which represent the patch length in bytes

Previous approaches like CANINE (Clark et al., 2022) were early demonstrations that a pretrained encoder could operate on character sequences without an explicit vocabulary. Newer models showed that this could also be done for generation models in ByT5 (Xue et al., 2022). In addition, the entropy patching method we describe is critical, which was introduced in a work from this same lab the year prior with MegaByte (Yu et al., 2023). However, we choose BLT for its recency and its contextualization within the LLM-dominant field, demonstrating competitiveness with today's models.

In order to perform the trials necessary to experiment with improving the language modeling performance of the BLT, we needed more compute than Google Colab would allow. Thus, we trained models with an approximate usage of 375 H100 hours across all our runs (successful and unsuccessful) using Prime Intellect.

We needed to scale down both the parameter size and the amount of data used in the training of our model. We scaled down the model from 8B parameters to 86M. We also computed the optimal number of tokens for training using the Chinchilla scaling law, approximating this count as 1.7B tokens (86M params * 20 = 1.7 B tokens). We source this data from FineWeb. We used an entropy model using a threshold which patches our bytes with an average patch size of 8. Figure 1 from the paper provides a heuristic to calculate the appropriate size of traditional language model to compare with, landing around 1.7x the size of the baseline BPE tokenized model.

For completeness, we also describe the setup for our training infrastructure. Because we did not have persistent hardware to use, and we did not have the budget to run instances continuously, we set up a .sh file to resolve the many dependencies (e.g. downloading the correct CUDA version, copying everyone's public keys into the cluster for shared ssh access). We used two persistent filestores to which we downloaded our training data and created a shuffled train/val/test split. We also had automated scp and teardown scripts for our compute-equipped machines upon job completion.

We also were careful to decide what to train from scratch and what to reproduce from scratch. Reading the codebase, we settled on reproducing a subset of the files, given that reproducing the entire working codebase, complete with distributed training scripts and utilities, would unnecessarily burden the project with systems implementations. We focus on the core functionality of the model:

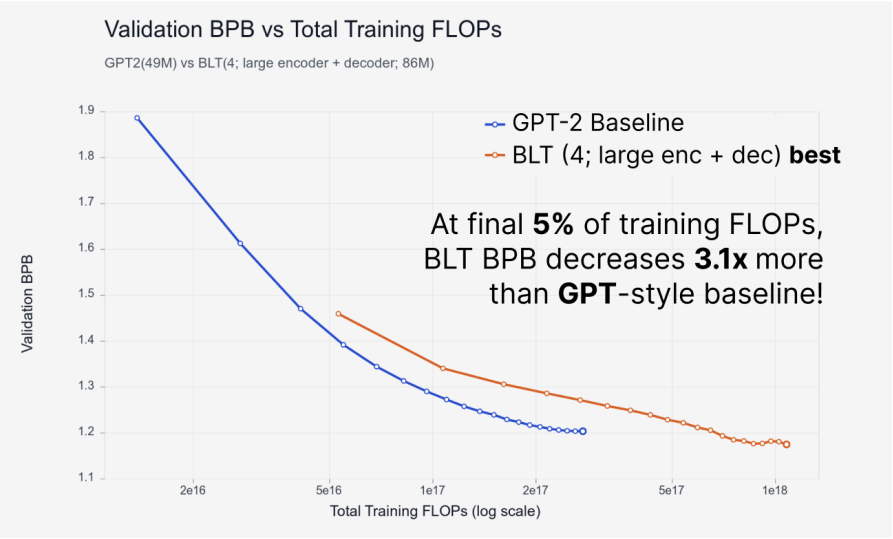
- bytelatent/model/blt.py
- bytelatent/model/local_models.py
- bytelatent/model/latent_transformer.py
- bytelatent/base_transformer.py

These four files make up the architecture of the Byte Latent Transformer in its entirety, and are the bulk of the reference repo's logic.

In addition to this code, both for the practical setup of our training environment and the BLT itself, we also created scripts for training our GPT-2 style baseline.

Due to the significantly smaller scale of our reimplementation, we were unable to identify the point at which a crossover occurs where the validation BPB ends up lower for the BLT model than the GPT-2 sized model. Instead, we measure the decrease in BPB for the final 5% of training FLOPs and measure the mean decrease

between each time-step. In addition to the BLT model's final BPB being lower than the GPT-2 sized model, the BLT achieved both a lower final BPB and a steeper continued improvement trend towards the end of training, suggesting that the crossover behavior seen in Figure 1 of the original paper would likely emerge given additional compute. While the GPT-2 model ends with BPB 1.204, ours ends with 1.175.



Our finding demonstrates the better BPB scaling of a BLT.

We enumerate the findings for our investigations of the three main questions mentioned prior:

1. How to reallocate parameters for ultra-low scales

We identify a "bottleneck challenge" which happens when attempting to reallocate parameters to different components of the architecture. Overall, these results demonstrate a natural finding that architectures at ultra-low parameter scales benefit from parameter consolidation, indicating that there may **exist a minimum effective encoder/decoder,** and that **we cannot rely on naively scaling down the same proportions provided by the paper.**

2. Improving byte representations

We find that when producing byte-gram embeddings to enrich byte representations with context, it is less useful to use 3, 4, 5-gram embeddings with smaller hash embedding tables than to use only a 4-gram embedding with a hash embedding table with a larger size.[1] Our hypothesis for why the additional embedding information fails to enrich the representations is that the three different embedding tables are not sufficiently large to learn meaningful representations at such an ultra-low scale.

3. Improving patch representations

We experiment with training using a sinusoidal patch length embedding, which we find improves performance over our baseline. Naturally, the length of an entropy patch provides an additional source of information regarding the underlying byte sequence, at least we hypothesize.

[1] We also experiment with learning weights for each of these context representations when adding them to the native byte representation, but we realize that this learned scaling could also be trivially learned by the embedding lookup table itself. For curious readers, the weights for 3, 4, and 5-byte-grams converged to ~(0.34, 0.42, 0.41) after approximately 600M tokens of training, suggesting that the redundancy of the 3-byte-gram is recognized by this training.

Overall, we hope to continue this work by scaling up runs on larger models, and potentially extending the training run of GPT-2 to observe the overtake point. We would also revisit the parameter reallocation experiments with a more principled search strategy. Our current ablations sampled a small number of configurations; a systematic sweep over encoder/decoder parameter ratios, potentially using a smaller proxy model to guide the search, would let us make stronger claims about the minimum effective component size.

Our repo link: https://github.com/laerdon/blt-replication

References

Clark, J., Garrette, D., Turc, I., and Wieting, J. 2022. CANINE: Pre-training an efficient tokenization-free encoder for language representation. *Transactions of the Association for Computational Linguistics*, 10:73–91.

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., van den Driessche, G., Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., Rae, J. W., Vinyals, O., and Sifre, L. 2022. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*, volume 35. NeurIPS 2022.

Pagnoni, A., Pasunuru, R., Rodriguez, P., Nguyen, J., Muller, B., Li, M., Zhou, C., Yu, L., Weston, J., Zettlemoyer, L., Ghosh, G., Lewis, M., Holtzman, A., and Iyer, S. 2025. Byte Latent Transformer: Patches scale better than tokens. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8365–8404. arXiv:2412.09871.

Penedo, G., Kydlíček, H., Ben allal, L., Lozhkov, A., Mitchell, M., Raffel, C., Von Werra, L., and Wolf, T. 2024. The FineWeb datasets: Decanting the web for the finest text data at scale. In *Advances in Neural Information Processing Systems*, volume 37. NeurIPS 2024. arXiv:2406.17557.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., and Sutskever, I. 2019. Language models are unsupervised multitask learners. *OpenAI Blog*, 1(8).

Sennrich, R., Haddow, B., and Birch, A. 2016. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30. NeurIPS 2017.

Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., and Choi, Y. 2019. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4791–4800, Florence, Italy.

Xue, L., Barua, A., Constant, N., Al-Rfou, R., Narang, S., Kale, M., Roberts, A., and Raffel, C. 2022. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306.

Yu, L., Simig, D., Flaherty, C., Aghajanyan, A., Zettlemoyer, L., and Lewis, M. 2023. MEGABYTE: Predicting million-byte sequences with multiscale transformers. In *Advances in Neural Information Processing Systems*, volume 36. NeurIPS 2023. arXiv:2305.07185.